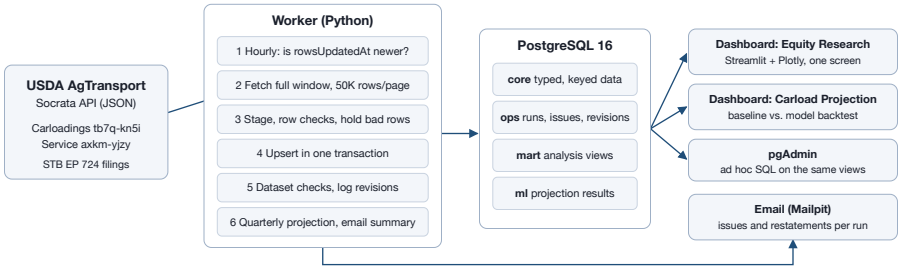
U.S. Rail Freight Monitor

Write-up for Jain Global Case Study · Andrew You · October 2026

1. Approach

The goal was a system an equity researcher can trust every Thursday without checking the plumbing. I built it in four layers: ingestion, data checks, storage and analysis. Everything starts with one command, `docker compose up`, and needs no accounts or keys.



System design. All five services (Postgres, worker, dashboard, pgAdmin, Mailpit) run locally from one Docker Compose file.

- **Ingestion.** The worker pulls both datasets from the public USDA API. The first load backfills all history, from the first filing (March 25, 2017). Every refresh re-fetches January 1, 2021 onward (five full years; the start moves up each January) and inserts or updates rows without deleting any, so corrections to recent weeks are picked up and older weeks are kept.
- **Scheduling.** Every hour the worker reads each dataset's public metadata, which includes `rowsUpdatedAt`, the time USDA last changed its rows. It loads only if that is newer than the timestamp saved with the last successful load, and loads anyway if nothing has succeeded in 7 days.
- **Data checks.** Seven checks guard each load. A failing row, such as an unknown railroad code, is not published: it is saved with its reason and original data in an issues table, listed in the run's email, and the last good value stays. Once fixed, it publishes on the next load. Problems with a whole dataset, such as a missing railroad, are flagged and emailed.
- **Dashboard.** One screen answers the brief's three questions: what is moving, what is driving the change, and whether service is improving.
- **Machine learning.** A separate page projects each railroad's quarterly carloads and tests whether a gradient boosting model, trained only on earlier quarters, beats a 13-week run-rate: carloads reported so far, plus the rest of the quarter at last year's level grown by the latest 13 weeks' year-over-year change. Trained on history from 2017, the run-rate still won (2.35% vs. 2.48% mean error over 30 quarters), so the page uses it.

2. Challenges

Challenge	How I addressed it
The datasets end their weeks on different days (Saturday and Friday).	They are joined on the Wednesday filing date, which both share.
The service dataset repeats the carloads data (157,226 of 157,228 values match).	I load carloads from one source only and use the copy as a cross check. A disagreement is flagged.
CP and KCS merged into CPKC in May 2025.	Volumes are summed into one continuous CPKC series. Speed and dwell start as a new series. Speed is an average, so CP's and KCS's cannot simply be added like carloads: blending them needs train-miles and train-hours the data doesn't include, and traffic once handed between the two now moves as one network.
Service metrics can't be compared across railroads.	Units match (mph, hours), but each railroad defines them differently in its filing to the regulator (Norfolk Southern's dwell leaves out coal trains and containers). I tried two workarounds. A car-cycle measure built from counts (cars online ÷ carloads per day) caught the 2022 eastern crisis, but its levels still reflect network shape. The share of cars idle 48 hours differed sevenfold between CSX and BNSF. So each railroad is compared only to its own history.
Railroads correct past weeks.	Every load re-pulls the full window, and each correction is logged with its old and new value.
Blanks, zeros and half cars in the source.	Blanks load as missing, never zero (4,355 values). Real zeros stay zeros (2,733 rows). Carloads allow decimals because CPKC reports half cars.
Holidays distort weekly comparisons.	The five lowest weeks all fall at Christmas or New Year. Each week is compared with the week exactly 52 weeks earlier, which keeps holidays aligned.

3. Assumptions

- USDA's republication matches the railroads' filings to the STB, and the latest published value is correct.

- History starts with the first carloads filing (March 2017, 499 weeks). Restatements older than five years are rare, so those weeks are loaded once and kept.
- Originated carloads measure network volume, because received carloads count interchanged cars twice.
- The user is one analyst on one machine, and a local inbox stands in for a real mail server.

4. How I Expect an Analyst to Use the System

Railroads file on Wednesday, USDA republishes Thursday morning, and the dashboard updates within the hour. The analyst opens one screen and sees whether volume is up, which commodities and railroads drove the change, and whether each railroad's service is improving against its own record. The quarter-to-date tracker and Carload Projection page project quarterly volume ahead of earnings. Exports feed models and decks, and the weekly email flags any corrected weeks.

5. Self-Assessment

Metric	Value	Metric	Value
Setup from a fresh clone	1 command	History loaded	499 weeks
Rows fetched per refresh (5 years)	986,349	Rows published (carloads, service)	157,228 / 1,334,178
First load time (carloads, service)	16 s / 198 s	Database size	265 MB
Lag after USDA publishes	≤ 1 hour	Data checks	7
Automated tests on real Postgres, in CI	24	Rows held in the latest load	0
Terminals in latest week on the map	64 of 64	Code (Python and SQL, excluding tests)	~1,900 lines

Strengths

ENGINEERING

- **Bad data cannot overwrite good data.** Every row is validated before the dashboard can see it. A failing row, such as an unknown railroad code, is set aside and the last good value stays. A load that fails partway is undone in full.
- **Runs from scratch with one command.** A new machine with Docker loads 9.5 years of data in about 4 minutes with no accounts or keys, then refreshes itself within an hour of each USDA update.
- **Small enough to review and extend.** About 1,900 lines of code, with 24 automated tests on every change. A new metric is one row in a reference table, not new code.

BUSINESS

- **Prevents the mistakes raw data invites.** The public files make three errors easy: counting a car twice when it changes railroads, ranking railroads on service metrics each defines differently, and losing CP and KCS history at the CPKC merger. The pipeline's rules prevent all three: only originated carloads count as volume, CP and KCS are stitched into CPKC, and service is compared only within each railroad.
- **Answers the brief's three questions on one screen.** Volume, its drivers and service trends sit together, one day after railroads file. Quarterly volume is projected weeks before earnings.
- **Changes to history are visible.** Railroads often correct past weeks. Each correction is logged and emailed, so the analyst knows when a number in their model has changed.

Weaknesses

ENGINEERING

- **No test for implausible values.** The checks catch structural problems such as unknown codes or a missing railroad. A value that is well formed but unlikely, such as volume falling 60% in a week from a filing error, would still publish.
- **No snapshots of past data.** Corrections are recorded only from the first run, so the system cannot show the data as it stood on a past date.
- **No dashboard tests, and one user only.** A code change could break a chart unnoticed. The system runs on one machine with no login.

BUSINESS

- **Volume is not revenue.** The system has no link to pricing or reported financials.
- **The model did not beat the run-rate.** Projecting each railroad's quarterly originated carloads, its average error was higher in both backtests (2.09% vs. 1.75%, 15 quarters; 2.48% vs. 2.35%, 30). It helped mainly in shocks such as 2020.

What I would improve with more time

ENGINEERING

- Hold values outside each series' normal range before they publish.
- Keep a weekly snapshot, so projections are tested on what was known then.
- Compare service across railroads, starting from the car-cycle measure.
- Test every dashboard panel, and host it with a login for a team.

BUSINESS

- Bridge volume to revenue: projected carloads × each railroad's reported revenue per carload, by segment.
- Alert the analyst when service or volume moves outside its normal range.